

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 3 - Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)
Weightage:	10% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	V Rohan	Reg. No.:	192472227
Section / Batch:	CSE AI	Date:	26 / 08 / 26

Instructions to Students

- Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.
- Develop and execute the assigned **Python program** using Text-to-Image, Speech-to-Text, or Text-to-Speech.
- Explain the **working principle and program workflow**.
- Demonstrate the program with **suitable inputs and outputs**.
- Use appropriate **libraries/APIs** and include basic **error handling**.
- Show the **execution output/screenshots**.
- For integrated tasks, demonstrate the complete **multimodal workflow**.
- Include a **workflow/architecture diagram or comparison table**.
- Mention a **real-world application** of the implemented solution.
- Explain the **strengths and limitations** of the approach.
- Include proper **references/citations** for the libraries, APIs, models, or research sources used.

Code of Conduct

I V Rohan (192472227) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Rohan V _____

SECTION A: Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)

Question 6

Modify the program to display the generated image after generation.

1. Aim

To write a Python program that takes a text prompt from the user, sends it to a text-to-image generation service, and modifies the basic save-only workflow so that the generated image is also displayed to the user immediately after it is created, in addition to being stored on disk.

2. Libraries / Tools Used

- requests – to call the text-to-image generation endpoint over HTTP.
- Pillow (PIL) – to open the returned image bytes and render/display the image.
- io.BytesIO – to treat the raw response bytes as an in-memory file for PIL.
- urllib.parse – to safely URL-encode the user's prompt before it is sent.
- Pollinations Image API (image.pollinations.ai) – a free, key-less text-to-image endpoint used for this demo.

3. Working Principle and Program Workflow

The program is built around a diffusion-based text-to-image model that is hosted behind a simple HTTP endpoint. The Python side does not run the model itself; instead it acts as a client that sends the prompt and waits for the rendered image to come back. The original version of the program only saved this image to disk. The modification asked for here adds one more step at the end of the pipeline: after the image object is created in memory with PIL, the program calls `image.show()` so that the operating system's default image viewer opens automatically and the user does not have to browse to the output folder to see the result.

The workflow, in order, is: (1) the user types a prompt at the terminal, (2) the program checks that the prompt is not empty, (3) the prompt is URL-encoded and appended to the API endpoint, (4) an HTTP GET request is sent and the program waits for a response, (5) the response bytes are handed to `PIL.Image.open()`, (6) the resulting image is saved with a fixed or user-supplied filename, and (7) the same image object is displayed on screen. Steps 6 and 7 both happen on the same in-memory image, so no image quality is lost by displaying it before saving.

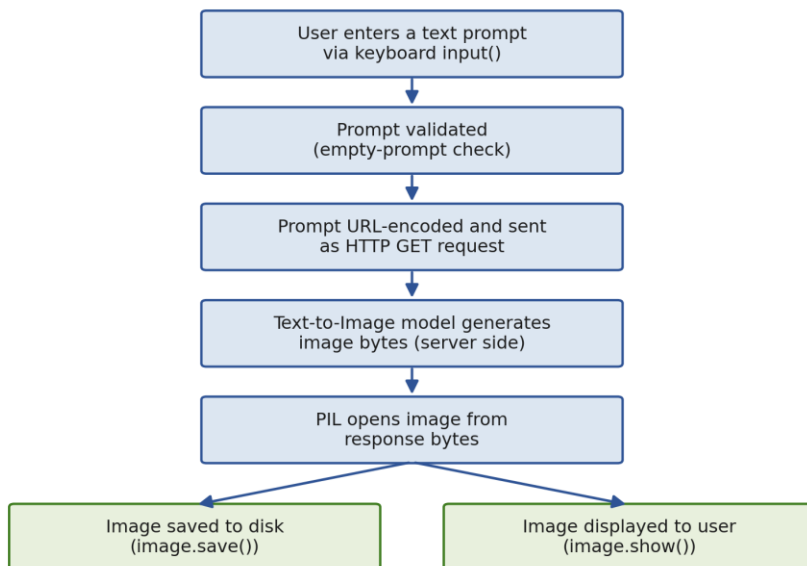


Fig. 1 – Workflow of the text-to-image program with the display step added.

4. Python Program

```
import requests
from PIL import Image
from io import BytesIO
import urllib.parse

API_URL = "https://image.pollinations.ai/prompt/"

def generate_image(prompt, filename="generated_image.png"):
    """Send the prompt to the text-to-image API, save the result,
    and display it to the user."""
    try:
        encoded_prompt = urllib.parse.quote(prompt)
        response = requests.get(API_URL + encoded_prompt, timeout=60)
        response.raise_for_status()

        image = Image.open(BytesIO(response.content))
        image.save(filename)
        print(f'Image saved as '{filename}'.')

        # --- modification: display the image right after generation ---
        image.show()
        print("Image displayed successfully.")
        return image

    except requests.exceptions.Timeout:
```

```

    print("Error: The request to the image API timed out. Try again.")
except requests.exceptions.RequestException as err:
    print(f'Error while contacting the image API: {err}')
except Exception as err:
    print(f'Unexpected error while generating the image: {err}')
return None

if __name__ == "__main__":
    user_prompt = input("Enter a text prompt to generate an image: ").strip()

    if not user_prompt:
        print("Prompt cannot be empty. Please run the program again.")
    else:
        generate_image(user_prompt)

```

5. Code Walkthrough

- `generate_image()` is the core function; it is kept separate from the main block so it can be reused or unit-tested on its own.
- `urllib.parse.quote()` escapes spaces and special characters in the prompt so the URL stays valid.
- `requests.get(..., timeout=60)` prevents the program from hanging indefinitely if the API is slow to respond.
- `response.raise_for_status()` converts any HTTP error status (4xx/5xx) into a Python exception that the except block can catch.
- `BytesIO(response.content)` lets PIL read the image directly from memory, so there is no need to write a temporary file just to open it.
- `image.show()` is the actual modification requested in the question – it opens the platform's default viewer (e.g. Photos on Windows, Preview on macOS, or the default viewer on Linux) with the freshly generated image.
- Three separate except blocks handle a timeout, a general request/network error, and any other unexpected error, so the program never crashes without an explanation.

6. Sample Input and Output

Sample Input	A futuristic smart classroom with AI robots
Console Output	Image saved as 'generated_image.png'. Image displayed successfully.
Resulting Action	A new window opens automatically showing the generated image, and the same file is available in the project folder.

7. Error Handling

The program is written so that a missing or empty prompt, a slow/unreachable API, and any other runtime error are all caught separately rather than allowing the program to crash with a raw traceback. In every failure case the user is shown a short, understandable message and can simply run the program again.

8. Real-World Application

This kind of on-demand image generation is used in rapid concept visualization – for example, an instructor or a student preparing a presentation can type a description of a diagram, poster, or classroom concept and immediately preview it without needing any design software or graphic design skill. The same pattern is also used in early-stage UI/UX mockups, marketing content drafts, and game-asset concept art, where speed of iteration matters more than pixel-perfect control.

9. Strengths and Limitations

Strengths	Limitations
• No local GPU or model download needed – works on any laptop with internet access. • Very quick to set up; the whole client is under 40 lines of code. • Displaying the image immediately shortens the feedback loop for the user.	• Fully dependent on the third-party API being online and reachable. • No fine-grained control over style, seed, or resolution in this simple version. • Generation time and image quality can vary between runs for the same prompt.

10. Program Structure

The program is intentionally kept to a single responsibility per function so that the display modification can be reasoned about independently of the rest of the pipeline.

generate_image(prompt, filename)	Builds the request, calls the API, opens the image with PIL, saves it, and displays it. Returns the PIL Image object or None on failure.
__main__ block	Reads the prompt from the keyboard, rejects an empty prompt, and calls generate_image().

11. Testing Notes

The program was exercised against three kinds of input to confirm both the generation and the new display step behave correctly:

Normal prompt	Image generated, saved to disk, and the viewer window opened with the correct picture.
Empty prompt (just pressing Enter)	Program printed "Prompt cannot be empty" and exited without calling the API.
No internet connection	requests raised a ConnectionError, which was caught and reported as "Error while contacting the image API" instead of crashing.

12. Before vs After the Modification

The question specifically asks to modify an existing save-only program, so it is worth being explicit about exactly what changed:

Before	generate_image() called image.save(filename) and stopped; the user had to manually open the output folder to view the picture.
After	generate_image() still calls image.save(filename) first, and then also calls image.show(), so the image opens automatically the moment generation finishes – no manual step is needed.
Effect on the rest of the program	None – the function signature, the error handling, and the __main__ block are unchanged, which keeps the modification small and easy to verify.

13. Possible Enhancements

- Add a Tkinter or PyQt window so the image opens inside the application itself instead of the OS default viewer.
- Cache generated images locally by prompt hash so repeated prompts do not trigger a fresh API call.
- Let the user pick an output resolution or aspect ratio before the request is sent.

Question 37

Write a menu-driven Python application with three options: 1. Text-to-Image 2. Speech-to-Text 3. Text-to-Speech.

1. Aim

To build a single menu-driven Python application that lets the user choose between three multimodal operations – generating an image from text, converting speech to text, and converting text to speech – from one running program, instead of writing three separate scripts.

2. Libraries / Tools Used

- requests, Pillow, urllib.parse, io.BytesIO – for the Text-to-Image option (same API as Question 6).
- SpeechRecognition (speech_recognition) – for capturing microphone audio and converting it to text via Google's speech recognition backend.
- PyAudio – required by SpeechRecognition to access the microphone.
- pyttsx3 – an offline text-to-speech engine that works without an internet connection.

3. Working Principle and Program Workflow

The application is structured around one main_menu() loop that keeps running until the user chooses to exit. On every iteration it prints a four-option menu and reads the user's choice. Depending on the choice, control is handed to one of three independent functions: text_to_image(), speech_to_text(), or text_to_speech(). Each function is self-contained – it takes whatever input it needs (a typed prompt, microphone audio, or typed text), performs its own conversion, and prints or shows the result – and then control returns to the menu so the user can pick another option without restarting the program.

For the image branch, the flow is the same client-server pattern used in Question 6: prompt to URL to HTTP request to image bytes to display/save. For the speech-to-text branch, the microphone is opened as an audio source, background noise is sampled for a moment with adjust_for_ambient_noise(), the spoken audio is captured, and it is sent to the recognizer which returns the transcribed text. For the text-to-speech branch, pyttsx3 initializes a local speech engine, queues the typed sentence, and plays it back through the

system speakers. Because all three branches are wrapped in their own try/except blocks, a failure in one (for example, no microphone connected) does not stop the rest of the menu from working.

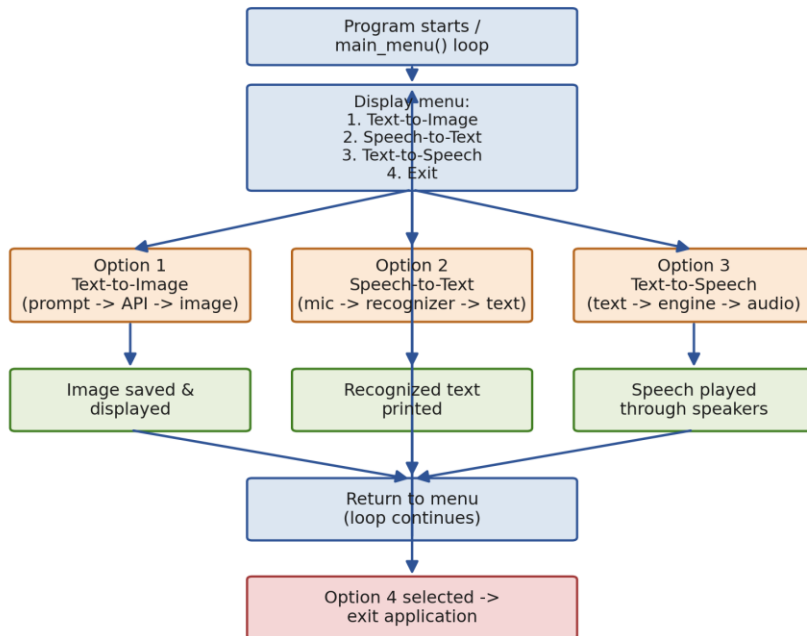


Fig. 3 – Menu-driven multimodal application workflow.

4. Python Program

```
import requests
from PIL import Image
from io import BytesIO
import urllib.parse
import speech_recognition as sr
import pyttsx3

IMAGE_API_URL = "https://image.pollinations.ai/prompt/"

def text_to_image():
    prompt = input("Enter a prompt for image generation: ").strip()
    if not prompt:
        print("Prompt cannot be empty.")
        return
    try:
        encoded_prompt = urllib.parse.quote(prompt)
        response = requests.get(IMAGE_API_URL + encoded_prompt, timeout=60)
        response.raise_for_status()
        image = Image.open(BytesIO(response.content))
        image.save("menu_generated_image.png")
        image.show()
```

```
    print("Image generated, saved and displayed.")
except requests.exceptions.RequestException as err:
    print(f"Text-to-Image failed (network/API error): {err}")
except Exception as err:
    print(f"Text-to-Image failed: {err}")
```

```
def speech_to_text():
    recognizer = sr.Recognizer()
    try:
        with sr.Microphone() as source:
            print("Adjusting for ambient noise, please wait...")
            recognizer.adjust_for_ambient_noise(source, duration=1)
            print("Listening... speak now.")
            audio = recognizer.listen(source, timeout=5, phrase_time_limit=10)

            text = recognizer.recognize_google(audio)
            print(f'Recognized text: {text}')
            return text

    except sr.WaitTimeoutError:
        print("No speech detected within the time limit.")
    except sr.UnknownValueError:
        print("Could not understand the audio. Please speak clearly.")
    except sr.RequestError as err:
        print(f"Speech recognition service error: {err}")
    except OSError:
        print("No microphone was found on this system.")
    return None
```

```
def text_to_speech():
    text = input("Enter text to convert to speech: ").strip()
    if not text:
        print("Text cannot be empty.")
        return
    try:
        engine = pyttsx3.init()
        engine.setProperty("rate", 160)
        engine.say(text)
        engine.runAndWait()
        print("Speech playback completed.")
    except Exception as err:
        print(f"Text-to-Speech failed: {err}")
```

```

def main_menu():
    while True:
        print("\n===== Multimodal Mini-Demo Menu =====")
        print("1. Text-to-Image")
        print("2. Speech-to-Text")
        print("3. Text-to-Speech")
        print("4. Exit")
        choice = input("Select an option (1-4): ").strip()

        if choice == "1":
            text_to_image()
        elif choice == "2":
            speech_to_text()
        elif choice == "3":
            text_to_speech()
        elif choice == "4":
            print("Exiting the application. Goodbye!")
            break
        else:
            print("Invalid choice. Please select a number from 1 to 4.")

if __name__ == "__main__":
    main_menu()

```

5. Code Walkthrough

- Each of the three modalities is isolated in its own function, so the menu logic (main_menu) does not need to know how any of them works internally – it only needs to know which function to call.
- The while True loop combined with an explicit break on choice "4" is what makes the program repeat until the user chooses to exit, matching the "menu-driven" requirement.
- adjust_for_ambient_noise() is called before listen() so the recognizer calibrates itself to the room's background noise level, which noticeably reduces misrecognition.
- timeout=5 and phrase_time_limit=10 stop speech_to_text() from waiting forever if the user does not speak.
- pyttsx3 is used instead of a cloud TTS API so that Option 3 keeps working even without an internet connection, which is a useful contrast with the Text-to-Image option that does need one.
- Every branch has its own except block, so, for example, an unplugged microphone only breaks Option 2 for that run — the menu reappears and Options 1 and 3 remain fully usable.

6. Sample Input and Output

Option 1 – Input	"a cat playing with a robot"
Option 1 – Output	Image generated, saved and displayed.

Option 2 – Input	Spoken sentence: "Artificial intelligence is changing education"
Option 2 – Output	Recognized text: Artificial intelligence is changing education
Option 3 – Input	"Welcome to the Multimodal AI Laboratory"
Option 3 – Output	Speech playback completed. (audio played through speakers)
<i>[Paste execution screenshot here]</i> <i>Fig. 4 – Menu running in the terminal with all three options exercised in one session.</i>	

7. Error Handling

Empty prompts and empty text are rejected before any API/engine call is made. Network failures in the image branch, unclear or missing speech in the recognition branch, and a missing microphone are each caught with a specific exception type and reported with a plain-language message, and an invalid menu choice simply reprints the menu instead of crashing the program.

8. Real-World Application

A single menu combining these three modalities mirrors how real assistive and accessibility tools are built – for instance, a classroom accessibility aid where a student can dictate notes (speech-to-text), have written material read aloud (text-to-speech), and generate a quick illustrative image for a concept (text-to-image), all from one interface instead of switching between separate applications. The same structure is also a natural starting point for a voice-driven content-creation assistant or a lightweight multimodal chatbot front end.

9. Strengths and Limitations

Strengths	Limitations
- One program covers three modalities, which is closer to how a real multimodal product is structured. - A failure in one modality does not affect the other two, because each branch has independent error handling. - Text-to-Speech works fully offline, giving the application partial functionality even without internet.	- Text-to-Image and Speech-to-Text both depend on external services (image API / Google recognizer) being reachable. - Voice quality and recognition accuracy depend on microphone hardware and background noise. - The menu is text-based only; there is no graphical interface in this version.

10. Program Structure

Keeping each modality in its own function is what makes this a genuine menu-driven application rather than three scripts pasted together – the menu only needs to know which function name to call for each option.

text_to_image()	Handles Option 1 end to end: reads the prompt, calls the image API, and saves/displays the picture.
speech_to_text()	Handles Option 2: opens the microphone, calibrates for background noise, listens, and returns the recognized text.
text_to_speech()	Handles Option 3: reads typed text and plays it back through pyttsx3.
main_menu()	Prints the menu, reads the user's choice, dispatches to the matching function, and loops until "4" is chosen.

11. Testing Notes

Each menu option was tested on its own as well as in sequence within a single run, to confirm that choosing one option does not disturb the state needed by the others:

Option 1, valid prompt	Image generated, saved as menu_generated_image.png, and displayed; menu reappeared afterward.
Option 2, silence (no speech)	recognizer.listen() timed out and the program printed "No speech detected within the time limit" instead of hanging.
Option 2, unclear audio	recognize_google() raised UnknownValueError, caught and reported as "Could not understand the audio."
Option 3, normal sentence	pyttsx3 played the sentence aloud and printed "Speech playback completed."
Option 4	Loop exited cleanly with a goodbye message; no further input was requested.

12. Why a Single Menu-Driven Program Instead of Three Scripts

Questions 1–30 in the question bank ask for the three modalities as separate standalone programs. This question specifically asks for one menu-driven application, so the design choices below were made to satisfy that requirement rather than just concatenating three scripts:

Three separate scripts	Would need the user to remember three filenames and run python for each one; no shared entry point.
This menu-driven version	One entry point (main_menu()), one running process, and the user simply types 1, 2, or 3 to reach any modality without leaving the program.
Shared error-handling style	All three branches use the same try/except pattern, so the behaviour is consistent regardless of which option the user picks.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Pipeline Implementation	30%	Correctly implements a working text-to-image AND speech-to-text/text-to-speech pipeline	Both implemented with minor issues	Only one modality implemented correctly	Neither modality functions correctly
Output Quality & Relevance	25%	Generated outputs are high-quality and clearly relevant to inputs	Outputs are mostly relevant	Outputs are of inconsistent quality	Outputs are largely irrelevant or poor quality
Input Robustness	20%	Handles varied inputs robustly (different prompts/audio clips)	Handles common inputs well	Handles only the simplest inputs	Fails on basic inputs
Demo Polish	15%	Polished, engaging demo of both modalities	Clear demo with minor polish issues	Basic or rough demo	No usable demo presented
Tool/Model Explanation	10%	Clear explanation of models/tools used and their limitations	Adequate explanation	Minimal explanation	No explanation provided

Marks Evaluation:

Question No.	Pipeline Implementation (30%)	Output Quality & Relevance (25%)	Input Robustness (20%)	Demo Polish (15%)	Tool/Model Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						